# CSE321 - Assignment #1

Ho Mai Phuong (20232006)
UNIST
South Korea
phuonggii@unist.ac.kr

## 1 PROBLEM STATEMENT

The objective of this assignment is to implement and analyze three tree-based index structures: the B-tree, B*-tree, and B+-tree. While these structures share a logarithmic theoretical complexity, they employ distinct architectural strategies regarding data storage, node occupancy, and rebalancing policies. This report seeks to investigate the practical performance trade-offs between these variants—such as how parameter tuning of the node degree ($d$) affects execution time, space utilization, and structural stability. By evaluating operations including insertion, point searching, range queries, and random deletion, this study aims to identify the optimal scenarios for each index structure in real-world data environments.

Implementation for this assignment can be found at GitHub[1].

## 2 FUNDAMENTAL INDEX STRUCTURE

### 2.1 B-tree

A B-tree is a self-balancing search tree data structure designed to optimize data access, handle large amount of data efficiently. It maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. Unlike standard binary trees, B-tree can have more than 2 children per node, allowing the height to be reduced.

In B-tree, fan-out is large. Because each node contains many keys and pointers to many children, tree's height is low, thus B-tree is shallow, reducing search to only 2-3 I/O operations, making it ideal for disk-based storage.

A B-tree of order $m$ satisfies the following properties:

- Every node has at most $m$ children.
- All non-leaf nodes (except root node) should have at least $m/2$ children.
- The root node has at least two children unless it is a leaf.
- All leaves appear on the same level.
- A non-leaf node with $k$ children contains $k-1$ keys.

Le $h$ be the height of the B-tree, $n$ be the number of entries in the tree and $m$ be the order of the tree.

Height when B-tree is completely full, or best-case height of B-tree is:

$h_{min} = \lceil \log_m(n+1) \rceil - 1$

Height when B-tree is least-filled (define $d = \lceil \frac{m}{2} \rceil$) , or worst-case height of B-tree is:

$h_{max} = \lfloor \log_d \left(\frac{n+1}{2}\right) \rfloor$

Some operations of B-tree are as followed:

(1) Insertion and splitting
- All insertions start at leaf node.
- Find the appropriate leaf node using the search logic and insert the key in sorted order.
- If the node is full, it splits:
  - The median (middle) key is identified.
  - Values less than the median are put in the new left node, and values greater than the median are put in the new right node.
  - The median key is pushed up into the parent node.
- Time complexity: $O(logn)$

(2) Search
- Starting at the root, compare the target key with the keys stored in the node.
- Continue until the key or a leaf node (meaning the key isn't in the tree).
- Time complexity: $O(logn)$

(3) Deletion
- If the key is in a leaf and deleting it doesn't violate the minimum occupancy, simply remove it.
- Internal Deletion: If the key is in an internal node, replace it with its in-order predecessor (the largest key in its left subtree) or successor, then delete that key from the leaf.
- Rebalancing
  - Borrowing: Taking a key from a sibling node via the parent.
  - Merging: Combining the underfilled node with a sibling and "pulling down" a key from the parent
- Time complexity: $O(logn)$

## 3 ADVANCED INDEX STRUCTURE

### 3.1 B+ tree

B+ tree is the extended version of B tree as there are some differences:

- Leaf nodes store all the key values and pointers to the actual data.
- Internal nodes store only the keys that guide searches.
- All leaf nodes are linked together using linked list, supporting efficient sequential and range queries.

(1) Search
- Navigate through internal nodes based on key comparisons, then reach the appropriate leaf node.
- Every search takes exactly the same amount of time because every piece of data is at the same depth.
- Time complexity: $O(logn)$

(2) Insertion
- Navigate to the correct leaf node and insert in the correct order.
- If overflow occurs, split the node, copy the median key to the parent (but it remains in the leaf node).
- Time complexity: $O(logn)$

(3) Deletion

[1] https://github.com/kirari804/UNIST-CSE-Assignment/tree/main/Database

- Finding and removing the key from the appropriate leaf node.
- Rebalancing (same as B tree):borrowing keys from siblings or merging nodes.
- Time complexity: $O(logn)$

(4) Range query
- Search for the first key stated in the range
- Follow the linked list pointers horizontally until the end of the range.
- Time complexity: $O(logn + k)$

Based on its characteristics, B+ Tree exhibits advantages:

- Efficient Range Queries: Linked leaves allow for O(k) sequential access.
- Cache Friendly: Smaller internal nodes allow more branching info in the CPU cache.
- Stable Latency: Search times are consistent across the dataset since every search must descend to the leaf level to retrieve the data.

However, there are some disadvantages:

- Wasted Space: Internal nodes store copies of keys already present in leaves.
- Full Path Requirement: Must always descend to the bottom, even for keys at the root.

### 3.2 B* tree

The B* Tree is a variant of the B-tree that further optimizes for space efficiency and reduced disk I/O. The most significant difference between a B-tree and a B* Tree is the minimum occupancy requirement. In B* tree, each node (except the root) must be at least 2/3 full. Therefore, B* tree is flatter and wider than the standard B-tree, thus leading to lower height. Thanks to this feature, there are some advantages when using the B* Tree structures:

- Space Optimization: 2/3 occupancy ensures minimal dead space in nodes.
- Lower Height: Denser nodes result in a flatter tree.

(1) Insertion
- When a node is full:
  - Redistribution: If the sibling has room, the B* tree moves some of the current node's keys to the sibling through the parent.
  - The 2-to-3 Split: Only if the sibling is also full does a split occur. However, instead of one node splitting into two, the two full nodes are split into three nodes, each 2/3
- Time complexity: $O(logn)$

(2) Deletion
- When a node falls below the 2/3 occupancy threshold, it first tries to "borrow" keys from its neighbors.
- A merge only occurs when the collective occupancy of the siblings cannot maintain the 2/3 requirement, triggering a 3-to-2 merge.
- Time complexity: $O(logn)$

When analyzing the above operations, B* exhibits disadvantages compared to other structures:

- Insertion Latency: Redistribution logic requires more memory writes and execution time than simple splitting.
- Implementation Complexity: 2-to-3 split logic is harder to maintain.

## 4 EXPERIMENTAL RESULT AND ANALYSIS

### 4.1 Insertion and Parameter Tuning

In the first experiment, 100,000 records were inserted into an empty tree. Overall, the **B+ Tree** is dynamically efficient (fastest), while the **B* Tree** is structurally stable and space efficient.

| Degree ($d$) | Variant | Time (s) | Splits | Utilization (%) |
|---|---|---|---|---|
| d = 3 | B Tree | 0.3810 | 26,096 | 63.85 |
| | B* Tree | 0.5095 | 20,079 | 82.98 |
| | B+ Tree | 0.3733 | 29,855 | 68.77 |
| d = 5 | B Tree | 0.3101 | 14,938 | 66.92 |
| | B* Tree | 0.4403 | 11,950 | 83.64 |
| | B+ Tree | 0.3076 | 16,366 | 69.70 |
| d = 10 | B Tree | 0.3239 | 7,273 | 68.70 |
| | B* Tree | 0.4300 | 5,911 | 84.53 |
| | B+ Tree | 0.3097 | 7,662 | 69.87 |

**Table 1: Insertion Performance Comparison (100,000 Records)**

(1) Insertion time

According to **Table** 1, we can observe that in terms of speed: *B+ Tree (Fastest) » B Tree » B* Tree (Slowest)*

- The **B+ Tree** consistently outperforms the others as expected. Even though it generates the most splits, B+ Tree only inserts data into the leaf level. The internal nodes only maintain keys and pointers that guide searches to the appropriate leaf nodes, which speeds up the time spent finding the insertion point.
- The **B* Tree** is the slowest. This is because instead of splitting a full node, it checks for available capacity in adjacent siblings and performs rotational balancing (*Redistribution*). This process requires more pointer updates and memory references than a simple split, which leads to an increase in execution time.
- Fan-out parameters
  - When degree is very large ($d = 10$), more keys are allowed per node, which significantly reduces height of the tree. While this theoretically improves performance, when reaching a node, a large horizontal scan (up to 19 keys) is conducted, which increases overhead.
  - When degree $d$ is small ($d = 3$), with only max 6 keys per node, the tree becomes very tall. Many splits are required (26096, 20079, 29855 respectively) to maintain the order criteria.
  - At $d = 5$, the tree height is low enough to be efficient, and the number of keys in each node (max 9) is small

enough that the CPU can scan them almost instantaneously. The amount of splits is moderate, further optimizing performance.

(2) Node splits
- As **Table** 1 shows, **B+ Tree** has the highest number of splits. This is because when a leaf node splits, the median key is pushed up to the parent, but a copy of it must stay in the leaf. This extra key storage fills up nodes faster than in a the standard B-Tree.
- The **B* Tree** has the lowest number of splits because it uses the *2-to-3* split strategy. When a node is full, it doesn't split immediately. It checks if its sibling has space. If it does, it performs a rotation. No new node is created; therefore, no split occurs. A split only happens when both a node and its sibling are completely full. Hence the B* Tree keeps its structure stable for longer.

(3) Node utilization
- **B* Tree** has the highest utilization, maintaining a massive 82.9% to 84.5% utilization. While standard B-Tree logic only guarantees 50% occupancy, the B* Tree's 2/3 rule ensures almost every node is nearly full, which is best optimized for space usage.
- **B-Tree**, **B+ Tree** : Both trees stay in the 63% to 69% range. While they are less efficient than the B* Tree, they still perform better than the theoretical 50% minimum.

## 4.2 Point Search

The experiment deals with searching for 10,000 random keys (a subset from Student ID) and measuring the mean execution time for each tree.

| Degree ($d$) | Variant | Mean Execution Time (s) |
|---|---|---|
| d = 3 | B Tree | 0.00000291 |
| | B* Tree | 0.00000296 |
| | B+ Tree | 0.00000250 |
| d = 10 | B Tree | 0.00000310 |
| | B* Tree | 0.00000326 |
| | B+ Tree | 0.00000255 |

**Table 2: Point Search Operations**

Overall, the **B+ Tree** records the lowest execution time:

- Since all actual data is stored in the leaf nodes, every search query has a predictable, uniform cost, requiring the same number of disk accesses.
- Because the data is moved entirely to the leaf level, the internal nodes are much smaller, allows the search algorithm to navigate through levels with fewer cache misses compared to the standard B-Tree.

The **B Tree*** demonstrates slightly higher latency. This is an interesting result because the B* Tree has lower height (flatter):

- Although the B* Tree has fewer levels to descend, each node is significantly denser. During a search, the algorithm performs more comparisons within each node before identifying the correct child pointer.
- When the CPU pulls a B* node into its cache, it pulls a denser block of keys and data. If a specific key is not in the cache line, the CPU has to fetch from RAM again.

## 4.3 Range Query

This experiment features a custom analytical query over a specific range of Student IDs. Specifically, it calculates the average GPA and height of male students whose IDs are between 202100000 and 202200001.

| Degree ($d$) | Variant | Execution Time (s) |
|---|---|---|
| d = 3 | B Tree | 0.013274 |
| | B* Tree | 0.012884 |
| | B+ Tree | 0.010859 |
| d = 10 | B Tree | 0.040403 |
| | B* Tree | 0.014546 |
| | B+ Tree | 0.010953 |
| **Common Results:** Avg GPA: 3.29 \| Avg Height: 174.08 | | |

**Table 3: Range Query Performance**

From **Table** 3, we observe that **B+ Tree** is the fastest, maintaining a near-constant execution time (around 0.0108 to 0.0109) regardless of degrees:

- Range queries are executed by performing a single point search to find the start of the range, then following the linked list pointers at the leaf level.
- The CPU simply reads contiguous leaf nodes, independent of the tree's height or degree once the initial leaf is found.

The **B* Tree** performs better than the standard B-Tree but could not match the B+ Tree:

- B* Tree is denser(2/3 full) and its height is lower than B Tree. Therefore for range query, a single node in B* Tree contains more of the target range than a 50%-full B-tree node, reducing the number of memory fetches and recursive calls.
- However, it lacks the linked-list leaves of the B+ Tree, so it must perform a recursive traversal.

The standard **B-Tree** exhibits the poorest performance:

- B-Trees store data in every node. To perform a range query, the algorithm implementation uses recursive in-order traversal.
- At $d = 10$, the B-Tree's time reaches 0.040403s (nearly 4x slower than B+ Tree). As nodes get wider, the complexity of managing the recursive path through many children per node increases exponentially, leading to performance degradation.

## 4.4 Deletion

The last experiment features the deletion of 2000 random records.

| Degree ($d$) | Variant | Mean Execution Time (s) |
|---|---|---|
| d = 5 | B Tree | 0.004073 |
| | B* Tree | 0.003273 |
| | B+ Tree | 0.003260 |
| d = 10 | B Tree | 0.004124 |
| | B* Tree | 0.003851 |
| | B+ Tree | 0.003759 |

**Table 4: Deletion Performance (2,000 Random Records)**

**B+ Tree** still shows the best performance, according to **Table** 4. This is because all data is in the leaf nodes, the algorithm always deletes from the bottom. There is no need for internal nodes swapping, making it the most efficient variant.

At $d = 5$, **B* Tree** deletion operation is slightly slower than B+ Tree. Because nodes are kept 2/3 full, underflow (where a node's number of keys drops below the minimum required threshold ) happens less frequently than in a standard B-Tree. Hence, there are fewer merging and borrowing operations.

The **B Tree** records the highest execution time:

- If the key is located in an internal node, the algorithm finds the in-order predecessor or successor, swap the keys, and then perform the deletion at the leaf level.
- If underflow occurs, B Tree performs rotation (borrowing from siblings). If both immediate siblings have only the minimum keys, merge the node with a sibling. This process increases the overall computational cost compared to other 2 trees.

Notice that as the degree increases to 10, the execution time for all three variants increase. This is because when a deletion triggers a borrow or merge operation, a higher degree means the algorithm has to shuffle more keys between siblings.

## 5 Additional Experiment

This experiment features **Sorted Insertion** of 100,000 records and measures the total execution time, node utilization. Multiple degrees are tested, varying from $d = 2$ to $d = 20$.

(1) Execution time
According to **Figure** 1, we can observe the low bottle neck and the optimal range for degree $d$:
- $d = 2$ to $d = 4$: At low degrees, the trees are excessively tall, requiring more node traversals and frequent memory allocations for new nodes. This results in the highest execution times across all variants, with the **B* tree** peaking at 0.4337s at $d = 2$.
- $d = 10$ to $d = 18$: This is the optimal degree range. As $d$ increases, the trees become flatter, significantly reducing traversal depth. The **B+ tree** achieves its peak performance at d=20 (0.1190$s$). The **B-tree** reaches a minimum at d=18 (0.1201$s$).

| Degree ($d$) | B-Tree | | B* Tree | | B+ Tree | |
|---|---|---|---|---|---|---|
| | Time (s) | Util (%) | Time (s) | Util (%) | Time (s) | Util (%) |
| 2 | 0.3543 | 50.00% | 0.4337 | 74.99% | 0.4307 | 50.00% |
| 4 | 0.2596 | 50.00% | 0.3205 | 62.51% | 0.2355 | 50.00% |
| 6 | 0.1669 | 50.01% | 0.2591 | 66.67% | 0.1682 | 50.00% |
| 8 | 0.1711 | 50.01% | 0.2486 | 68.75% | 0.1702 | 50.00% |
| 10 | 0.1427 | 50.01% | 0.2160 | 65.01% | 0.1405 | 50.01% |
| 12 | 0.1539 | 50.01% | 0.2133 | 66.68% | 0.1337 | 50.00% |
| 14 | 0.1257 | 50.01% | 0.2237 | 67.86% | 0.1468 | 50.01% |
| 16 | 0.1209 | 50.01% | 0.2021 | 65.64% | 0.1230 | 50.02% |
| 18 | 0.1201 | 50.01% | 0.1947 | 66.66% | 0.1199 | 50.01% |
| 20 | 0.1377 | 50.02% | 0.1894 | 67.51% | 0.1190 | 50.01% |

**Table 5: Sorted Insertion Performance**

- At $d = 20$, B+ Tree and B* Tree reach their the minimum execution time. For the **B-tree**, execution time begins to climb (0.1377$s$). This is due to the overhead of scanning and shifting keys within a massive node. While the B* Tree also has this disadvantage, its nodes are fuller, hence this tree has lower height, which doesn't increase the execution time ar $d = 20$.

(2) Node utilization
- The 50% utilization: Both **B-tree** and **B+ tree** maintain a near-constant utilization of 50.01%. In sorted insertion, a node split occurs at the rightmost edge, so the left sibling is exactly half-full. These left nodes are never revisited to be filled further.
- Time-Space tradeoff: While the **B* tree** is the most space-efficient, it is the slowest variant. At $d = 10$, for example, the B* tree (0.2160$s$) is about 50% slower than the B+ tree (0.1405$s$). This shows the computational overhead required to balance keys between siblings rather than simply splitting nodes.

Overall, from this experiment, we can see that:

- **B+ Tree** demonstrates the best overall scalability for sorted insertion, reaching the lowest execution time at $d = 20$ since it splits only at the leaf level.
- **B* Tree** provides good memory density but at the cost of higher insertion latency. It is the optimal for environments where storage space is expensive.
- **B-tree** offers moderate efficiency in utilization and computational cost. However it shows earlier degradation at higher degrees compared to other structures.




**Figure 1: Execution Time vs. Degree for Sorted Insertion**